CircleGuard · Informe de la Parte 1 (pipeline CI/CD inicial)

Ingeniería de Software V · Universidad Icesi · Nov 8, 2026 · @MISAEL

Resumen y enlaces

El pipeline de CircleGuard lleva un commit de `main` a producción de forma automática, probada, versionada y con aprobación manual, y se demostró de punta a punta en AWS el 8 de noviembre de 2026 con dos releases: `v1.0.0` y `v1.1.0`.

Equipo: José Manuel Rojas (A00405392, `jmrojas06`, plataforma) y Pierre Cuevas (A00405468, `pierrecuevas`, aplicación y calidad). Nivel de uso de IAG: 3, colaboración con IAG.

Qué	Dónde
Repositorio	ingesoft5/circle-guard-finalproject-A00405392-A00405468
Tablero ágil	GitHub Projects #8
Jenkins	http://44.221.115.232:8080
SonarQube	http://44.221.115.232:9000
Nexus (registro Docker)	http://44.221.115.232:8081
Ambiente dev	http://34.230.234.121
Ambiente stage	http://3.209.207.195
Ambiente prod	http://50.19.116.114
Releases	v1.0.0 y v1.1.0
Evidencia por historia	`docs/evidencias/sprint-1/`

El docente tiene cuentas `docente` de solo lectura en Jenkins, SonarQube y Nexus. Sus usuarios y contraseñas están en el README del repositorio, en la sección «Acceso del docente».

Corrida real del pipeline en AWS

El 8 de noviembre `main` recorrió el pipeline completo dos veces: `v1.0.0` tardó unos 23 minutos de trabajo hasta stage, más 58 de espera de aprobación, y `v1.1.0` llegó a la pausa de prod en 9 minutos. Las cifras salen de las APIs de Jenkins, SonarQube, Nexus y GitHub, consultadas después de cada corrida.

Corrida	Disparo	Resultado	Duración	Qué demuestra
`main` #2 (`v1.0.0`)	*Build Now*	SUCCESS	81,6 min (58,3 de aprobación)	Pipeline completo hasta prod, aprobado por `pierrecuevas`
`PR-56` #1	*Webhook* al abrir el PR	SUCCESS	9,5 min	CI completa sobre los 9 servicios: 121 pruebas, SonarQube e integración
`main` #3 (`v1.1.0`)	*Webhook*, push a `main`	SUCCESS hasta stage	9 min hasta la pausa	Pipeline automático, 6 E2E en verde
`circleguard-rollback` #1	Manual (dev, versión `9.9.9`)	FAILURE esperado	10 s	Aviso de fallo por WhatsApp y Discord, dev intacto

Producción: `v1.1.0` se aprobó en prod (build `main` #3, 18:49) y el rollback de prod a `v1.0.0` se ejecutó con `circleguard-rollback` #2 (3 min 48 s, con confirmación manual y los 10 contenedores `healthy`); después prod volvió a `v1.1.0` con el build #3 del mismo job. Las capturas van en el Anexo B.

El detalle por etapa, con tiempos, está en `ensayo-pipeline-real.md`.

E1.1 Gestión ágil y branching

El trabajo se gestionó en un tablero vivo con 14 historias del Sprint 1 (61 puntos), PR revisados por pares y `main` protegida, y el sprint cierra con review y retrospectiva documentadas.

- **Marco y tablero:** Scrum con tres iteraciones (Sprint 0 el 4 de noviembre, Sprint 1 del 5 al 8 y Sprint 2 del 9 al 22). El tablero tiene los campos Status, Sprint, Story Points y Entregable, y cada historia lleva criterios de aceptación medibles. Al 8 de noviembre, los 61 puntos del Sprint 1 están en Done; HU-13, el cierre del sprint, se completa con la entrega de este informe en el repositorio.

- **Documentos del sprint:** `sprint-0.md` y `sprint-1.md`, con planning, review contra los criterios, métricas y retrospectiva de cada integrante. El burndown se toma a diario con `burndown.sh`.
- **Estrategia de branching:** GitHub Flow con Conventional Commits, documentada en `CONTRIBUTING.md`.
- **Protección de `main`:** un PR obligatorio con una aprobación de otro integrante, aprobaciones descartadas si llegan commits nuevos, la última persona que empuja no puede aprobar, reglas también para administradores, historia lineal, solo *squash* y conversaciones resueltas.
- **Revisión por pares:** 17 PR fusionados en los dos sprints, cada uno aprobado por el otro integrante. Las aprobaciones, con lo que cada revisor verificó, quedan en GitHub y en la bitácora de IAG.
- **Check de Jenkins en cada PR:** `continuous-integration/jenkins/pr-merge` se reporta en todos los PR y, desde el 8 de noviembre por la noche, es obligatorio en la protección de `main`.

E1.2 Infraestructura en la nube con Bash

Cuatro VMs en AWS se crean y configuran con scripts Bash idempotentes: `up.sh` reconstruyó toda la plataforma en 3,5 minutos el 8 de noviembre, sin pasos manuales.

VM	Tipo	Contenido	IP elástica
`circleguard-tools`	t3.xlarge (16 GB)	Jenkins, SonarQube y Nexus	44.221.115.232
`circleguard-dev`	t3.large (8 GB)	5 servicios núcleo, middleware y web	34.230.234.121
`circleguard-stage`	t3.large (8 GB)	igual que dev	3.209.207.195
`circleguard-prod`	t3.large (8 GB)	igual que dev	50.19.116.114

- **Ambientes aislados:** cada uno tiene su VM, su *security group* y su IP elástica.
- **Idempotencia demostrada:** la primera ejecución de `up.sh` tardó 2 min 45 s; la segunda, 1 min 20 s, con todo en «ya existe» (evidencia).
- **Sin configuración manual:** Jenkins se configura como código (usuarios, permisos, credenciales, *Shared Library* y jobs); SonarQube y Nexus, el repositorio Docker y el *webhook* de GitHub, por API.

- **Costos:** presupuesto mensual de USD 50 con alertas por correo; las VMs encendidas cuestan unos USD 0,44 por hora.
- **Trade-offs declarados:** el registro Docker va por HTTP dentro de la VPC (TLS queda para la Parte 2), la interfaz de Nexus y el SSH solo se abren a las IP administrativas, y el despliegue usa el usuario `ubuntu` con una llave dedicada.

E1.3 Empaquetamiento inmutable

Las 9 imágenes (8 servicios y el cliente web) son inmutables: *multi-stage*, usuario no root, con *healthcheck* y etiquetadas con la versión semántica y el commit, nunca con `latest`.

- **Construcción:** los servicios pasan de `gradle:8.14-jdk21` a `eclipse-temurin:21-jre-alpine` (99 a 153 MB); el web, de `node:22-alpine` con `expo export` a `nginx-unprivileged:1.27-alpine` (22 MB), que además hace de *reverse proxy* hacia los servicios.
- **Registro:** repositorio Docker `circleguard` de Nexus con política `allow_once`, así que un *tag* publicado no se puede sobrescribir. El release `v1.0.0` dejó las 9 imágenes con los *tags* `1.0.0` y `sha-f2b7707` (18 componentes), y `v1.1.0` agregó los suyos.
- **Compose por ambiente:** `base.yml` más un override para dev, stage y prod, con *healthchecks* y `depends_on: service_healthy`. Los tres ambientes corren 10 contenedores `healthy` con la misma imagen; solo cambian `IMAGE_TAG` y la configuración.
- **Secretos:** se generan una vez en cada VM con `openssl rand` y nunca pasan por el repositorio ni por Jenkins.

E1.4 Pipeline Jenkins (CI)

Jenkins ejecuta la CI de cada PR por *webhook*, con una *Shared Library*: commitlint, build, pruebas con JaCoCo, SonarQube con *quality gate* bloqueante e integración, solo sobre los servicios afectados.

- **Multibranch y *webhook*:** el job `circleguard` descubre `main` y cada PR. Las entregas de GitHub (`push` y `pull_request`) respondieron HTTP 200 y arrancaron los builds solas.
- **Shared Library:** 11 pasos reutilizables en `jenkins/vars/` (`changedServices`, `commitLint`, `runGradle`, `sonarScan`, `semanticRelease`, `dockerPublish`, `deployCompose`, `smokeTest`, `e2eTests`, `releaseHistory` y `notifyTeam`), documentados en `jenkins/README.md`.
- **Ejecución selectiva:** un PR sobre `gateway` construyó solo ese servicio; un cambio común (Gradle, `Jenkinsfile` o `jenkins/`) construye los 9.

- **Quality gate bloqueante:** el PR #47, de demostración, fue bloqueado por el *quality gate* y GitHub registró el estado `error`, con aviso por WhatsApp y Discord.
- **CI completa sobre el código final:** el build de `PR-56` duró 9,5 min: build 1,0, pruebas con JaCoCo 3,6, SonarQube con *quality gate* 2,4 e integración 2,2. Ejecutó 121 pruebas sin fallas y el *quality gate* «CircleGuard» dio OK en los 8 proyectos de SonarQube.

E1.5 Entrega continua y promoción

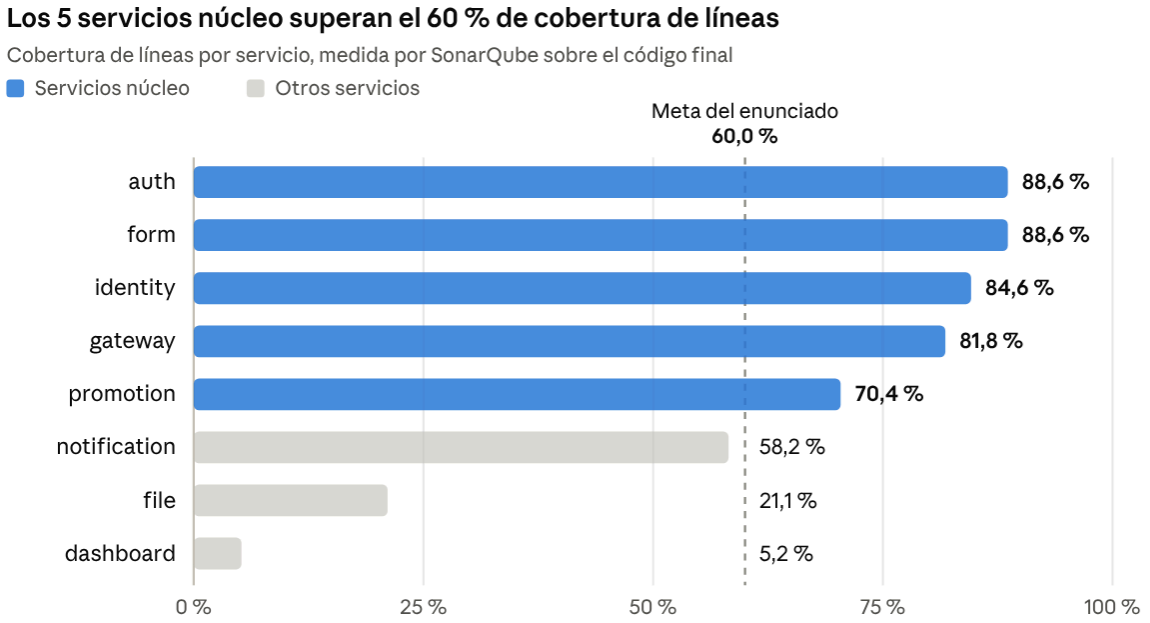
Cada versión de `main` se calcula, se publica y se promueve sola a dev y stage, y llega a prod solo con aprobación manual; el rollback consiste en volver a desplegar una versión anterior.

1. **Release:** semantic-release lee los Conventional Commits, calcula la versión (un `feat` sube la MINOR, un `fix` la PATCH) y crea el *tag* `vX.Y.Z` y la *GitHub Release* con sus notas.
2. **Package + Publish:** construye y publica las 9 imágenes en Nexus con ese *tag*.
3. **Deploy dev:** despliega por SSH y corre una prueba de humo (`/healthz` y el *readiness* de los 5 servicios núcleo).
4. **E2E contra dev:** Playwright corre dentro del pipeline; si falla, no se promueve.
5. **Deploy stage:** mismo despliegue y prueba de humo, y un aviso por Discord y WhatsApp pidiendo la aprobación.
6. **Aprobación para prod:** un `input` que solo pueden aprobar `jmrojas06` o `pierrecuevas`, con 2 horas de plazo. `pierrecuevas` aprobó `v1.0.0`.
7. **Deploy prod:** despliega y repite la prueba de humo.

- **Notificaciones de fallo:** Discord y WhatsApp avisan fallos, la espera de aprobación y el resultado. Se probó provocando un rollback de dev a una versión inexistente: falló en 10 s, el pipeline avisó por los dos canales y dev siguió intacto.
- **Rollback:** el job `circleguard-rollback` recibe `ENV` y `VERSION`, imprime el historial del ambiente, pide confirmación solo en prod y vuelve a desplegar esa versión. Las imágenes son inmutables y siguen en Nexus. Cada despliegue queda en `/opt/circleguard/RELEASES`.
- **Rollback de prod demostrado:** `circleguard-rollback` #2 (`ENV=prod`, `VERSION=1.0.0`) devolvió prod de `1.1.0` a `1.0.0` en 3 min 48 s, con confirmación manual y 10 contenedores `healthy`; el build #3 la volvió a `1.1.0`.

E1.6 Pruebas automatizadas

Las 121 pruebas automáticas (110 unitarias y 11 de integración) y los 6 E2E de Playwright pasan dentro del pipeline, con cobertura de líneas de 70 % a 89 % en los 5 servicios núcleo.



SonarQube · cobertura de líneas por servicio, 8 de noviembre de 2026

La meta de 60 % aplica a los 5 servicios núcleo; los otros tres se compilan, prueban y empaquetan ahora y se despliegan en la Parte 2.

- **Cobertura:** el build exige un mínimo de 60 % de líneas en `auth`, `identity`, `promotion`, `form` y `gateway` con `jacocoTestCoverageVerification`. Las cifras de abajo las midió SonarQube sobre el código final.
- **Integración con Testcontainers:** 11 casos en 3 flujos (`form` a Kafka a `promotion` con Neo4j y Redis; el QR de `auth` validado por `gateway` con Redis; `promotion` a Kafka a `notification`). Encontraron 3 errores reales del código base: el *gateway* dejaba pasar a personas sospechosas, `promotion` no actualizaba Redis si el usuario no tenía contactos y faltaba el serializador JSON de Kafka.
- **E2E con Playwright:** 6 pruebas sobre el cliente web (inicio de sesión válido y con clave incorrecta, cuestionario de salud, validación de acceso válida y falsa, y versión desplegada). Corren contra dev dentro del pipeline, publican un informe HTML y JUnit, y frenan la promoción si fallan. Al probar en un navegador real encontraron 4 errores de la aplicación, entre ellos un 403 por CORS en el login.
- **Lectura del panel de SonarQube:** su indicador «Coverage» combina líneas y ramas. Las ramas estaban bajas por el código que genera Lombok, y por eso `auth` mostraba 56,7 % frente a 88,6 % por líneas. Tras excluir ese código con `lombok.config` (PR

#59), SonarQube mide 90,6 % en `auth`, 91,0 % en `identity`, 75,7 % en `form`, 62,3 % en `promotion` y 79,2 % en `gateway`. La meta del enunciado es de líneas.

E1.7 Change management y release notes

Cada cambio sigue un proceso formal, con solicitud, clasificación y aprobación según su tipo, y cada versión sale con notas generadas automáticamente por el pipeline.

- **Política y plantilla:** `docs/change-management/` define los tipos de cambio (estándar, normal y emergencia), quién aprueba cada uno, el trazado hacia el PR y el *rollback*. La plantilla `solicitud-cambio` de GitHub usa la etiqueta `change-request`.
- **Uso real del proceso:** el RFC #52 cubrió el primer despliegue a producción de `v1.0.0`: aprobado por `pierrecuevas` en el `input` del build `main` #2, enlazado a las Releases y cerrado.
- **Release notes automáticas:** semantic-release publica la *GitHub Release* con las notas agrupadas por tipo (Funcionalidades y Correcciones), adjunta `CHANGELOG.md` como artefacto del build y comenta en cada PR incluido la versión en la que salió.
- **Esquema de etiquetado:** `vMAJOR.MINOR.PATCH` es a la vez el *tag* de Git, la *Release* y el *tag* de las 9 imágenes (más `sha-<commit>`). `v1.0.0` reunió los `feat` y `fix` acumulados del proyecto; `v1.1.0` salió de un solo `feat(web)`, el PR #57.

E1.8 Documentación, video e IAG

La documentación permite reproducir todo: un README operativo, diagramas de arquitectura y de pipeline, la referencia de la *Shared Library* y una bitácora de IAG con reflexión crítica de cada integrante.

- **README operativo:** la sección «Operación» del README da los requisitos, las URL de las herramientas y los tres ambientes, cómo llega un cambio a prod, cómo reconstruir la nube con `up.sh` y cómo hacer un rollback.
- **Diagramas:** `docs/arquitectura/` tiene tres diagramas Mermaid (servicios, eventos y datos; ambientes en AWS; y pipeline), dibujados a partir del código y no de memoria.
- **Referencia del pipeline:** `jenkins/README.md` explica cada etapa, cada paso de la librería, las credenciales y las reglas de concurrencia. Cada carpeta de infraestructura y despliegue tiene su propio README.
- **Bitácora de IAG:** `IAG_REGISTRO.md` declara el nivel 3 y registra, por entrada, el prompt, lo que produjo la IA, lo que se verificó o cambió y la revisión del otro integrante. Termina con la reflexión crítica de `pierrecuevas` y de `jmrojas06`.
- **Video del pipeline de punta a punta:** circleguard-pipeline-sprint1.mp4 (3 min 47 s), adjunto al release `v1.1.0`, con su guion y antes/después.

Decisiones, trade-offs e incidencias

Las decisiones de diseño buscaron un pipeline reproducible y seguro con pocas máquinas, y la corrida real dejó cuatro incidencias que se resolvieron y quedaron documentadas.

Decisiones de diseño

- **Sin `@semantic-release/git`:** `main` está protegida también para administradores y no se abrió una excepción para el *bot*; el `CHANGELOG.md` va a la *Release* y al artefacto del build.
- **`agent none` en el pipeline:** la espera de aprobación para prod no ocupa uno de los 2 ejecutores de Jenkins.
- **Las 9 imágenes con la misma versión:** el compose de cada ambiente usa un solo `IMAGE_TAG`.
- **Un build nuevo de `main` aborta el anterior** (`abortPrevious`), incluido el que espera la aprobación de prod; por eso no se fusiona nada mientras una versión espera el `input`.
- **En web no hay biometría:** el guardia pega el código QR o usa un lector USB.

Incidencias de la corrida del 8 de noviembre

Qué pasó	Causa	Qué se hizo
El primer build de `main` (#1) falló a los 17 s	Ocurrió durante el arranque de Jenkins, mientras `up.sh` reaplicaba su configuración	Se disparó el #2, que corrió limpio
El job `circleguard-rollback` no aparecía	`up.sh` construye Jenkins con la copia local, que estaba atrasada respecto a `main`	`git pull`, segundo `up.sh tools` y *Reload Configuration from Disk*
Las entregas del *webhook* dieron 502	Jenkins estaba apagado	Con Jenkins encendido responden 200
`main` omitió build, pruebas y SonarQube	La ejecución selectiva no vio servicios cambiados: el último commit era de documentos	El PR #56 tocó `jenkins/` (cambio común) y ejecutó la CI completa

Lecciones aprendidas y Sprint 2

Lo que más pesó fue tener todo como código y validar con datos reales: casi cada error de la corrida se encontró ejecutando y consultando las APIs, no leyendo.

- **Plataforma reproducible:** `up.sh` reconstruyó las 4 VMs y las 3 herramientas en 3,5 minutos, y la primera corrida real de `main` salió completa porque cada pieza se había validado antes en local.
- **Las pruebas de integración y E2E pagaron su costo:** encontraron 3 errores del código base y 4 de la aplicación que las pruebas unitarias no veían.
- **Dos lecciones sobre la CI:** varios PR se fusionaron mientras Jenkins estaba apagado, y la CI completa sobre el código final solo se ejecutó al final; los PR apilados costaron rebases y reaprobaciones.

Acciones para el Sprint 2: correr `git pull` antes de `up.sh`, activar el check obligatorio de Jenkins en `main`, abrir un PR a la vez contra `main` y correr los E2E en local antes de cualquier PR que toque el *frontend*.

El Sprint 2 (del 9 al 22 de noviembre) suma infraestructura como código con Terraform, Kubernetes para los 8 servicios, observabilidad con Prometheus, Grafana y Jaeger, patrones de diseño, seguridad y pruebas de carga con Locust y ZAP.

Anexos

Anexo A. Registro de uso de IAG. El equipo declara el nivel 3, colaboración con IAG. La bitácora completa, con el prompt, lo producido, lo verificado y la revisión de cada entrada, y la reflexión crítica de cada integrante, está en `IAG_REGISTRO.md`. Cada integrante puede explicar y modificar todo lo que entregó.

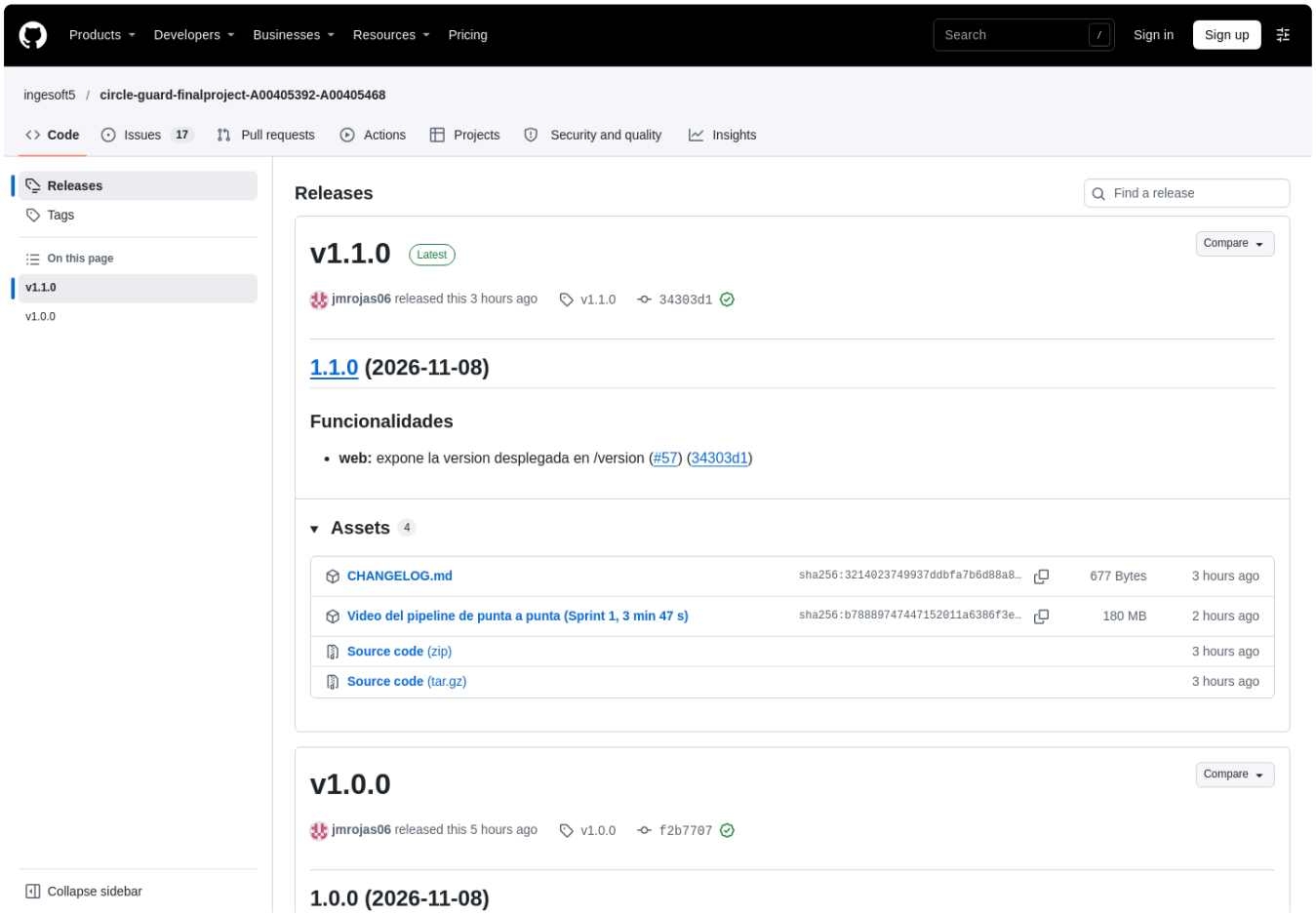
Anexo B. Capturas de pantalla *(se insertan aquí, en este orden)*

1. Jenkins: vista de etapas del build `main` #2 (`v1.0.0`) completo hasta `Deploy prod`.
2. Jenkins: la pausa en «Aprobación para prod» del build #3, con el botón «Desplegar en prod».
3. Jenkins: build `PR-56` #1 con las etapas de CI en verde y el informe de pruebas.
4. Jenkins: job `circleguard-rollback`, el build #1 fallido a propósito, el #2 (rollback de prod a `1.0.0`) y el #3 (prod de vuelta a `1.1.0`).
5. GitHub: *Releases* `v1.0.0` y `v1.1.0` con sus notas.
6. GitHub: *Settings* de *Webhooks*, «Recent Deliveries» con respuestas 200.
7. GitHub: protección de la rama `main`.
8. GitHub: el tablero del proyecto con las historias en Done.
9. SonarQube: lista de los 8 proyectos con su *quality gate*, y el panel de un servicio núcleo.
10. Nexus: el repositorio `circleguard` con las 9 imágenes y sus *tags*.
11. Consola de EC2 con las 4 instancias en ejecución.

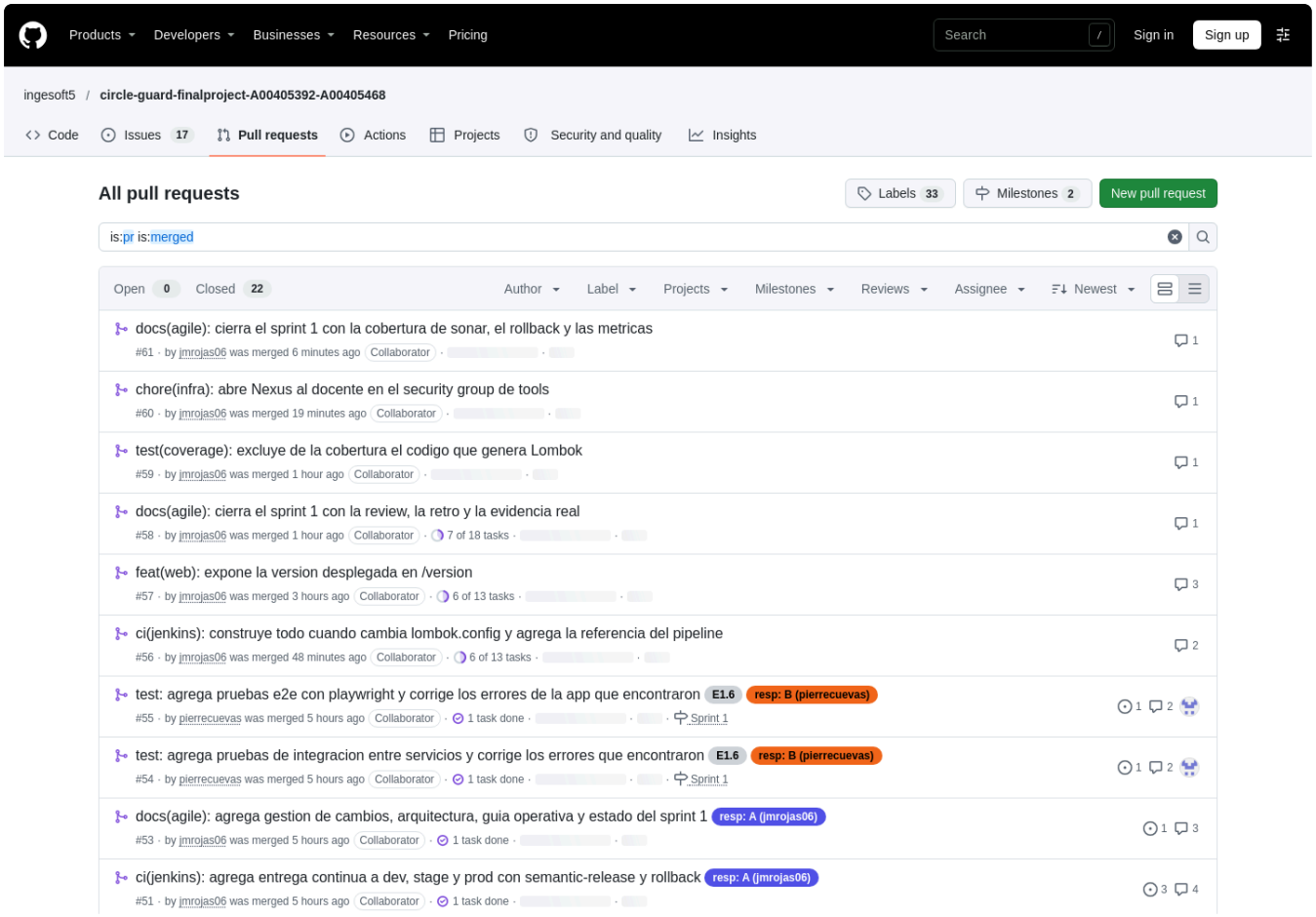
12. Los tres ambientes en el navegador y `curl` a `/version` en dev, stage y prod.
13. Discord y WhatsApp con el aviso del fallo provocado y el de aprobación.
14. Informe HTML de Playwright (`Playwright E2E (dev)`) y el de cobertura JaCoCo.

Capturas incluidas (8 de noviembre de 2026)

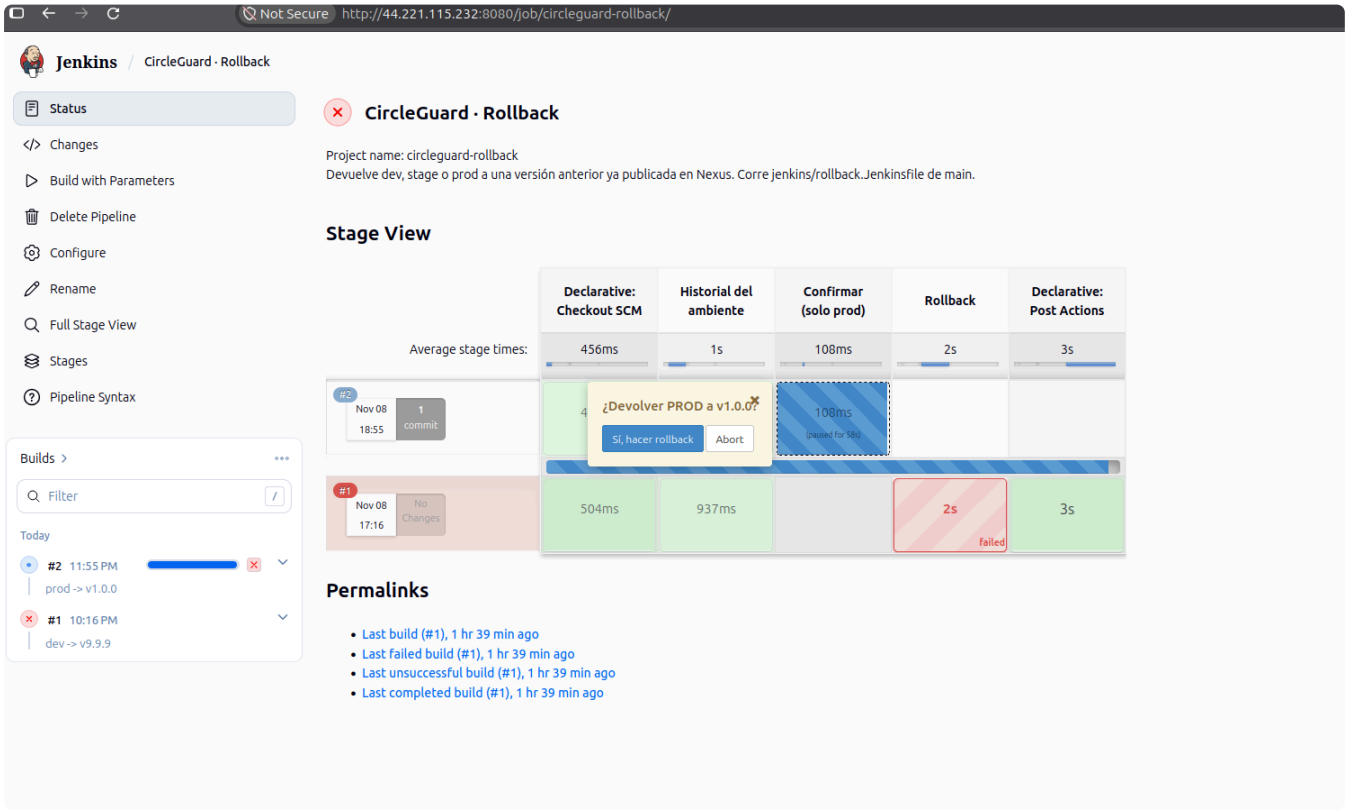
GitHub Releases: `v1.0.0` y `v1.1.0`, con sus notas automáticas, el `CHANGELOG.md` y el video del pipeline como adjunto.



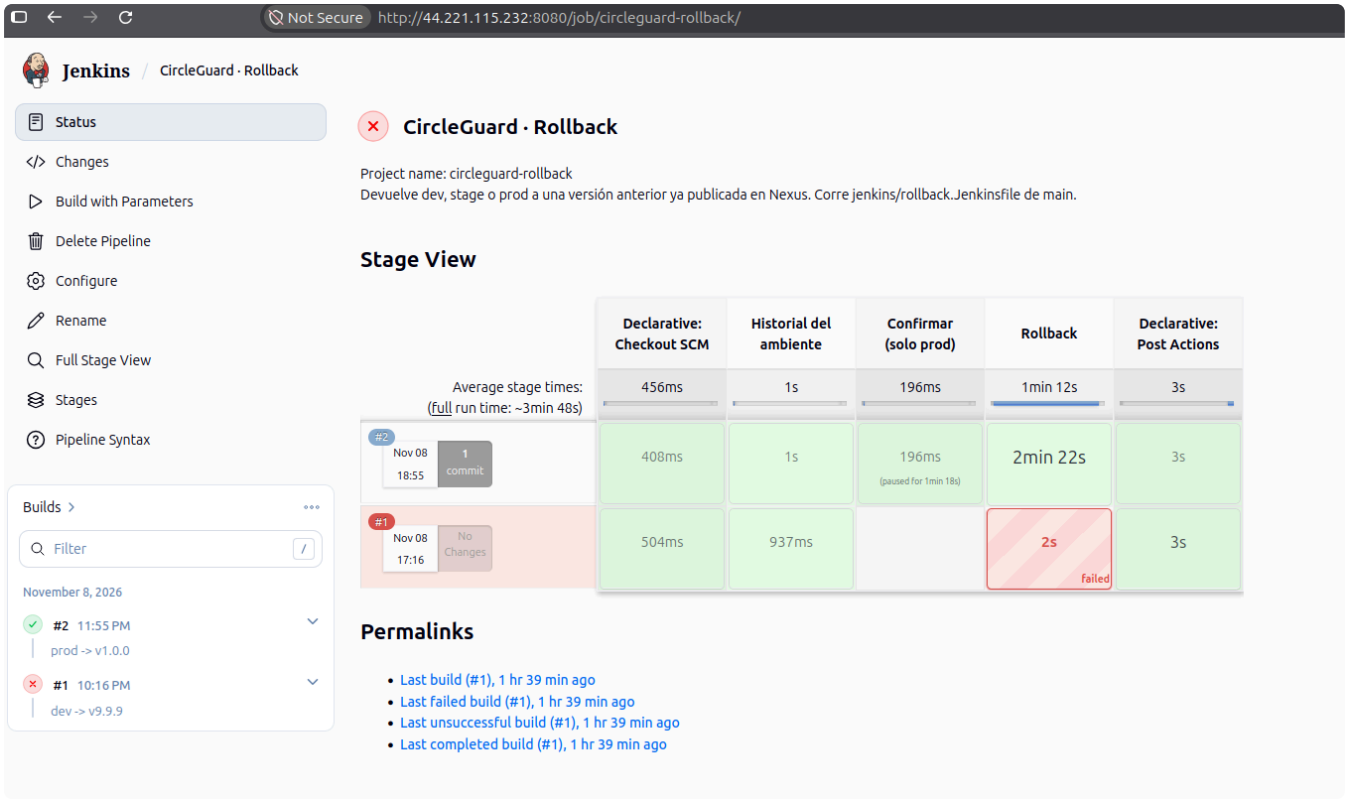
GitHub, pull requests fusionados: cada uno aprobado por el otro integrante.



Jenkins, job `circleguard-rollback`: el build #2 a la espera de la confirmación «¿Devolver PROD a v1.0.0?» (el #1 es el fallo provocado en dev).



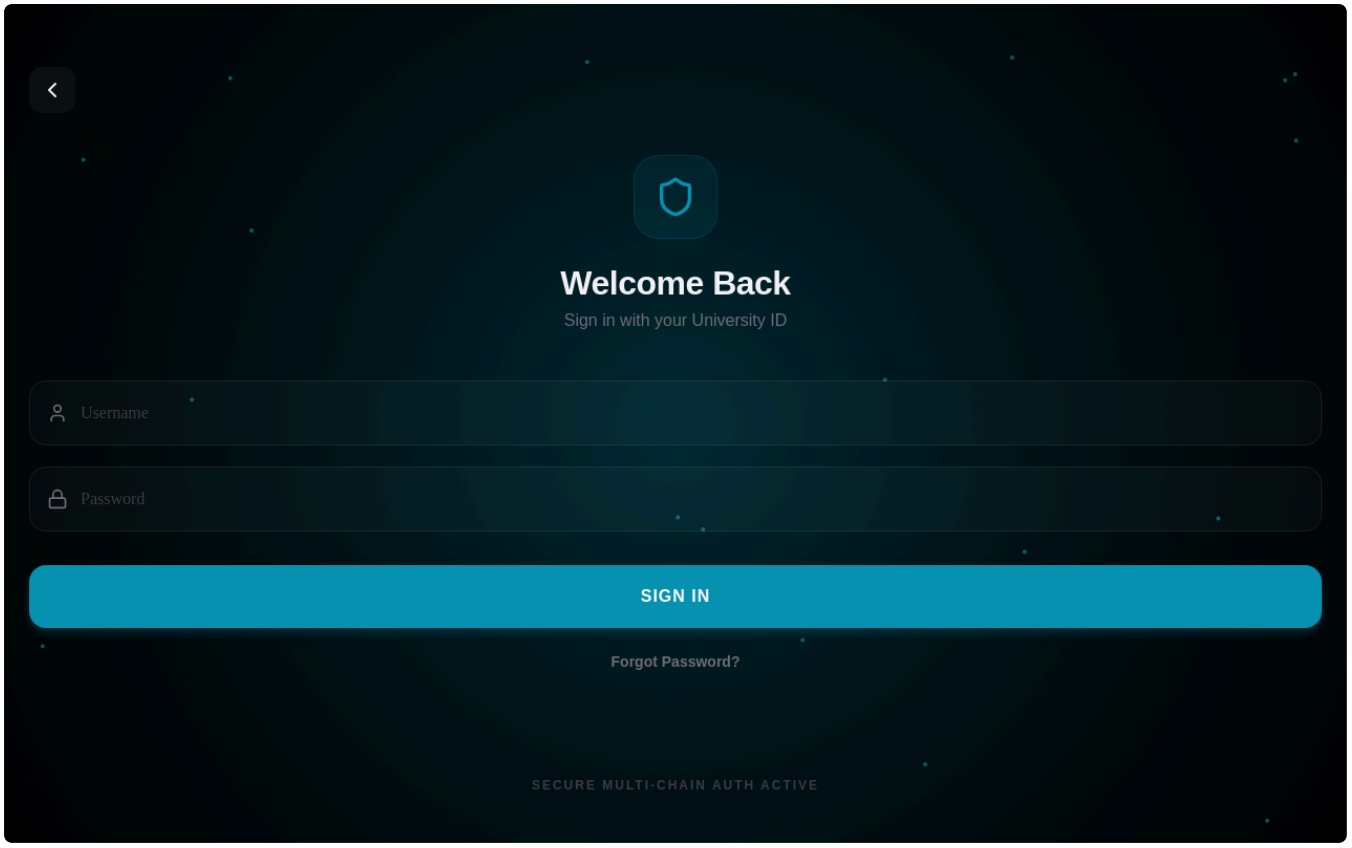
Jenkins, job `circleguard-rollback`: el build #2 terminado, con todas las etapas en verde.



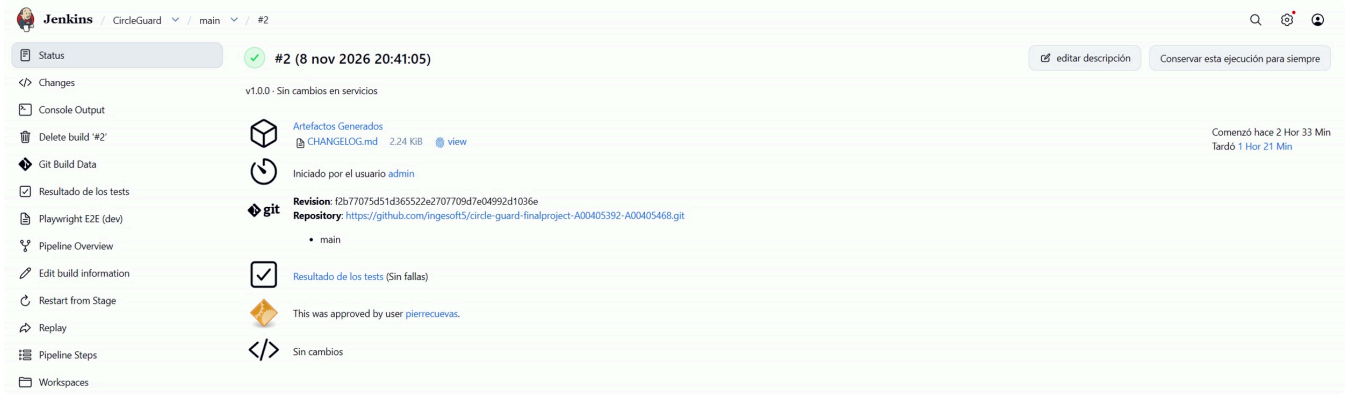
Los tres ambientes en `v1.1.0` (respuesta de `/version` en dev, stage y prod):

```
dev   http://34.230.234.121/version  ->  1.1.0
stage http://3.209.207.195/version   ->  1.1.0
prod  http://50.19.116.114/version   ->  1.1.0
```

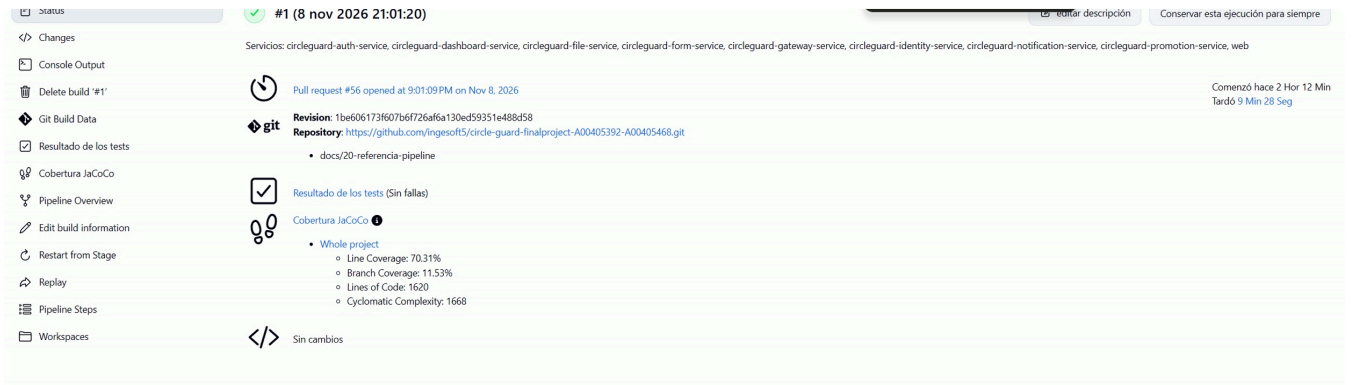
Aplicación web en producción: pantalla de inicio de sesión servida por nginx en `http://50.19.116.114`.



Jenkins, build `main` *#2 (* `v1.0.0` *):* terminado con éxito, con el `CHANGELOG.md` como artefacto, los tests sin fallas, el informe de Playwright E2E y la aprobación de prod por `pierrecuevas`.



Jenkins, build `PR-56` *#1:* CI completa de un PR sobre los 9 servicios, con tests sin fallas y cobertura JaCoCo de líneas de 70,31 %.



Nexus, repositorio `circleguard`*:* las imágenes de los servicios y, en `circleguard-web`, los tags `1.0.0`, `1.1.0`, `sha-f2b7707` y `sha-34303d1`.

